Universidad de los Andes Facultad de Ingeniería	**Ingeniería de Sistemas y Computación** **ISIS-1611: Inteligencia artificial** Semestre: 2026-2 Créditos: 3	Accredited ABET Engineering Accreditation Commission

Taller 3: Razonamiento bajo incertidumbre

Contexto general

La Amazonía atraviesa una temporada seca intensa. Una red de estaciones ambientales y cámaras térmicas vigila sectores de bosque expuestos a incendios. Sus observaciones son inciertas: la neblina puede parecer humo, una superficie expuesta al sol puede registrar una temperatura elevada y una falla del sensor puede activar una alarma.

Alerta Bosque combina estas señales para estimar la probabilidad de incendio y ayudar al centro de respuesta a decidir si conviene inspeccionar el sector, enviar una brigada o continuar el monitoreo.

Cuando se confirma una emergencia, un dron se desplaza hacia un punto de descarga de agente retardante. El viento puede desviarlo y algunas zonas presentan mayores costos o riesgos. El dron necesita una política que indique qué acción tomar en cada estado, considerando los posibles resultados de sus movimientos.

Las dos fases solo comparten el contexto y se resuelven de forma independiente. La probabilidad calculada con la red bayesiana no se usa en el MDP del dron.

Objetivos de aprendizaje

1. Calcular probabilidades conjuntas, marginales y condicionales, y aplicar normalización, la regla del producto y la regla de Bayes.
2. Implementar el cálculo de utilidad esperada y la selección de una acción.
3. Implementar inferencia por enumeración y compararla con eliminación de variables usando una implementación suministrada.
4. Implementar muestreo a priori y por rechazo, y estudiar el efecto de las muestras y de la evidencia.
5. Resolver un MDP mediante iteración de valores y extraer su política.
6. Explicar los resultados de inferencia y el efecto de variar el ruido, las recompensas y el descuento sobre los valores y la política.

Escenario del problema

Fase 1: diagnóstico probabilístico

Las variables son binarias: 1 indica presencia del evento o nivel alto, y 0 su ausencia o nivel bajo. Las probabilidades y utilidades son datos ficticios suministrados para el taller.

Cada archivo de `layouts/bayes` contiene los nodos en orden topológico, sus padres, sus tablas de probabilidad condicional (CPT) y los casos de evidencia. Cada entrada de una CPT es P(X=1|padres); el complemento corresponde a X=0. Los bits de la clave siguen el orden de la lista `parents`.

Escenario	Estructura y propósito
`sensor_basico`	Incendio → Alarma. Control de cuatro combinaciones para Bayes y utilidad esperada.
`cadena_humo`	Sequedad → Incendio → Humo. Independencia condicional y eliminación con factores pequeños.
`bosque`	Tormenta y Sequedad → Incendio; Incendio → Humo; Incendio y Sequedad → Temperatura; Humo, Temperatura y Falla → Alarma.

Control del punto 1

En `sensor_basico`, P(Incendio=1)=0.10, P(Alarma=1|Incendio=1)=0.90 y P(Alarma=1|Incendio=0)=0.05. El caso `alarma` fija Alarma=1; `sin_alarma` fija Alarma=0 y `sin_evidencia` no fija ninguna variable.

Acción	Sin incendio	Con incendio
monitorear	0	-100
inspeccionar	-8	-25
brigada	-30	-5

La utilidad esperada es EU(a)=`sum_s` P(s) U(a,s). Seleccione la acción de mayor utilidad esperada. En un empate conserve la primera acción en el orden de la tabla.

Casos de la red bosque

Además de `sin_evidencia`, se suministran alarma (Alarma=1), `humo_calor` (Humo=1, Temperatura=1), `alarma_falla` (Alarma=1, Falla=1) y `rara` (Tormenta=1, Falla=1, Alarma=1). La ventana permite consultar las CPT; los archivos JSON contienen sus valores completos.

Fase 2: respuesta secuencial

El dron se mueve sobre una cuadrícula. Su estado es la posición (x,y), con origen arriba a la izquierda. Las acciones se consideran en el orden N, E, S, W; este será también el orden de desempate.

El dron elige una dirección, pero el viento puede desviarlo hacia un lado. El parámetro `noise` es la probabilidad total de desviación: la dirección elegida tiene probabilidad 1-`noise` y cada dirección perpendicular tiene probabilidad `noise`/2. Si choca con un muro o borde, permanece en su celda. Las probabilidades que llevan al mismo sucesor se suman.

Símbolo	Significado
S	Posición inicial, transitable.
.	Celda transitable normal.
#	Muro.
G	Punto de descarga: terminal favorable.
X	Pérdida del dron: terminal desfavorable.
R	Zona transitable con penalización adicional.

El modelo entrega T(s,a,s') y R(s,a,s'). Al entrar en G o X se recibe la recompensa terminal en lugar de la recompensa por movimiento. La misión termina allí y V(terminal)=0: no se vuelve a recibir esa recompensa. En una celda no terminal se recibe la recompensa por movimiento (`living_reward`); si el sucesor es R se suma la penalización de riesgo (`risk_penalty`), incluso si el dron permanece en esa celda.

El descuento cumple $0 \leq$ `gamma` < 1. La solución es una política que maximiza la suma esperada de recompensas descontadas, no una ruta fija.

Mapa	Ruido	Paso	Descuento	Propósito
`control`	0	-0.04	0.90	Dos celdas: S y G.
`cuadricula`	0.20	-0.04	0.90	Rebotes, ciclos y terminales +1/-1.
`atajo`	0.20	-0.04	0.95	Descarga +2 y pérdidas -5 junto a la ruta corta.
`zona_riesgo`	0.10	-0.04	0.95	Descarga +2; R añade -0.40 al paso.

Los parámetros anteriores son los valores iniciales de cada mapa. Las opciones de ejecución permiten variarlos sin editar sus archivos.

Estructura del proyecto

El proyecto AlertaBosque entrega la carga de escenarios, los modelos, la ejecución, las métricas y las interfaces. Complete las funciones indicadas conservando sus firmas y formatos de retorno. Los docstrings incluyen instrucciones y ayudas; el marcador `# TODO: Add your code here` señala cada implementación.

Archivos que debe modificar

Archivo	Funciones	Punto
`algorithms/probability.py`	`normalize`, `expected_utility`, `best_action`	1
`algorithms/inference.py`	`enumeration_ask`	2
`algorithms/sampling.py`	`prior_sample`, `sampling_ask`	3
`algorithms/planning.py`	`action_value`, `value_iteration`	4
`algorithms/planning.py`	`extract_policy`	5

Módulos suministrados

Módulo	Uso
`models/bayes.py`	Carga la red, consulta CPT y construye la tabla conjunta para controles pequeños.
`models/elimination.py`	Implementación completa de eliminación de variables y traza de factores. Se usa para estudiar los factores intermedios.
`models/mdp.py`	Estados, acciones, transiciones, recompensas y simulación del dron.
`runner.py`	Coordina experimentos, referencia exacta, semillas e historiales.
`view/graphics_display.py`	Red y CPT, resultados de muestreo, valores, políticas y trayectoria.
`view/text_display.py`	Tablas y resultados en texto.
`menu.py` / `main.py`	Menú de terminal y ejecución por comandos.
`layouts/bayes` y `layouts/mdp`	Escenarios JSON y parámetros de referencia.

Complete únicamente los bloques `TODO` y conserve el código que está fuera de ellos.

Los puntos 2 y 3 reutilizan `normalize`. El punto 3 usa la enumeración del punto 2 como control exacto. El punto 5 reutiliza `action_value` y los valores del punto 4.

Ejecución

El proyecto requiere Python 3.11 o superior. Desde la carpeta del proyecto, instale la dependencia del menú y ejecute:

```
python -m pip install -r requirements.txt
```

```
python main.py
```

El proyecto puede ejecutarse mediante el menú principal o por línea de comandos. El menú permite seleccionar el ejercicio, escenario y parámetros correspondientes, y visualizar los resultados en modo gráfico o texto.

También puede ejecutar directamente un caso por comandos. Sin `-t` se abre la visualización gráfica; con `-t` los resultados se muestran en la terminal.

Opción	Uso / valor inicial
`-p / --point`	Punto 1 a 5. Inicial: 1. El punto 6 usa los resultados anteriores.
`-l / --layout`	Nombre del escenario, sin .json. Inicial: `sensor_basico` o `cuadricula`.
`--case / --query`	Caso: `sin_evidencia`. Consulta: Incendio.
`--evidence`	Sustituye el caso, por ejemplo Alarma=1,Falla=0.
`-a / --method`	Solo punto 3: `prior`, `rejection` o `elimination`. Inicial: `prior`.
`--order`	Eliminación: `forward`, `reverse` o lista de ocultas separadas por coma.
`-n / --samples`	Intentos de muestreo. Inicial: 10000.
`--repeat / --seed`	Repeticiones: 1. Semilla: 7; repetición i usa seed+i, desde i=0.
`--noise / --living-reward / --discount`	Sustituyen los parámetros del mapa MDP.
`--iterations / --tolerance`	Máximo de barridos: 1000. Tolerancia: 1e-8.
`--max-steps`	Límite de trayectoria: 100 pasos.

```
python main.py -p 1 -l sensor_basico --case alarma -t
```

```
python main.py -p 3 -l bosque --case rara -a rejection -n 10000 -t
```

```
python main.py -p 5 -l atajo --noise 0.3 -t
```

Ejercicios

Punto 1: Probabilidades y utilidad esperada (15 %)

Antes de enviar una brigada, el centro de respuesta necesita distinguir una alarma de un incendio confirmado. Use `sensor_basico` y las utilidades suministradas.

a) Para `sensor_basico`, construya manualmente las cuatro filas de P(Incendio,Alarma) usando la regla del producto. Compruebe que suman uno. Obtenga P(Alarma=1) sumando las filas correspondientes y calcule P(Incendio=1|Alarma=1) mediante la regla de Bayes. Incluya la tabla y los cálculos en el PDF.

b) Implemente `normalize` en `algorithms/probability.py`. Retorne un diccionario nuevo con las mismas claves y pesos normalizados. Rechace entradas vacías, pesos negativos o no finitos y suma cero. No modifique la entrada.

c) Implemente `expected_utility` y `best_action` en `algorithms/probability.py`. La primera recibe una distribución normalizada y las utilidades de una acción. La segunda retorna la acción seleccionada y un diccionario con la utilidad esperada de cada acción. Conserve la primera acción en los empates.

d) Ejecute `sin_evidencia`, `alarma` y `sin_alarma` y compare las acciones seleccionadas. Explique la diferencia entre probabilidad y utilidad esperada en el literal a del punto 6a.

Punto 2: Inferencia exacta por enumeración (20 %)

El centro incorpora varias observaciones y necesita responder consultas usando la estructura de una red bayesiana.

a) Complete el bloque de `enumeration_ask` en `algorithms/inference.py`. Recorra las variables en orden topológico, multiplique las probabilidades locales y sume sobre las variables ocultas. Normalice los dos pesos finales y retorne el diccionario {0: probabilidad, 1: probabilidad}. No modifique `evidence`. Si la consulta está observada, retorne una distribución puntual, siempre que la evidencia sea posible. Si la evidencia es imposible, lance `ValueError`.

b) Use `sensor_basico` con el caso `alarma` para verificar que `enumeration_ask` coincide con el resultado calculado manualmente en el punto 1. Luego, en `bosque`, calcule la probabilidad de incendio para `sin_evidencia`, `alarma` y `alarma_falla`. Incluya los tres valores en una tabla en el PDF y describa brevemente cómo cambia la probabilidad con cada evidencia.

c) En `cadena_humo`, use la estructura de la red para justificar que Humo es independiente de Sequedad dado Incendio. Verifique con su implementación P(Humo=1|Incendio=1,Sequedad=0) y P(Humo=1|Incendio=1,Sequedad=1). Incluya los resultados y la justificación en el PDF. Explique por qué esa igualdad no implica independencia sin observar Incendio.

Punto 3: Eliminación de variables e inferencia aproximada (20 %)

En este punto trabajará con eliminación de variables y muestreo. La primera reorganiza cálculos parciales; el segundo aproxima probabilidades a partir de muestras.

Eliminación de variables: estudio y comparación

a) En `cadena_humo`, calcule manualmente P(Humo=1) eliminando primero Sequedad y luego Incendio. Incluya en el PDF las variables y entradas de cada factor intermedio y el resultado final. Después, seleccione Eliminación de variables, consulte Humo sin evidencia y use el orden Topológico para comprobar el resultado.

b) En `bosque`, consulte Incendio con el caso `alarma`. Primero obtenga el resultado mediante enumeración en el punto 2. Luego, en el punto 3, seleccione Eliminación de variables y ejecute los órdenes Topológico (`--order forward`) e Inverso (`--order reverse`). Compruebe que las probabilidades obtenidas coinciden y compare el número de filas del mayor factor intermedio entre los dos órdenes de eliminación. Explique lo observado en el literal b del punto 6a.

Muestreo: implementación

c) Implemente `prior_sample` en `algorithms/sampling.py`. Genere una muestra completa de la red: asigne 0 o 1 a cada variable en orden topológico. Para cada variable X, consulte P(X=1|padres) usando los valores ya muestreados de sus padres. Asigne X=1 si `rng.random()` es menor que esa probabilidad y X=0 en caso contrario. Use exclusivamente el generador recibido.

d) Implemente `sampling_ask` en `algorithms/sampling.py`. Sin rechazo, estime una marginal a partir de todas las muestras y exija evidencia vacía. Con `reject=True`, descarte las incompatibles con la evidencia y normalice los conteos aceptados. El parámetro `samples` fija el número de intentos, no el número de aceptaciones. Retorne la pareja (distribución estimada, número de muestras aceptadas). Si no se acepta ninguna muestra, retorne (`None`, 0). Esto no significa que la evidencia sea imposible.

e) En `bosque`, consulte Incendio y pruebe muestreo a priori sin evidencia con 1000 y 10000 intentos. Luego pruebe rechazo con `alarma` y `rara` usando esas cantidades. Observe la aproximación al resultado exacto y la cantidad de muestras aceptadas. Conserve algunos resultados representativos para analizarlos en el literal c del punto 6a.

Como el muestreo es aleatorio, usar más intentos no garantiza menor error en una ejecución particular.

Punto 4: MDP e iteración de valores (20 %)

El dron necesita valorar sus movimientos considerando tanto el viento como las recompensas futuras. El modelo de transición se entrega implementado.

a) En `cuadricula`, identifique en el PDF los estados, acciones, transiciones, recompensas, estados terminales y descuento. Explique por qué la posición del dron, junto con la acción elegida, basta para determinar las probabilidades del siguiente estado.

b) Implemente `action_value` en `algorithms/planning.py` usando Q(s,a)=`sum_s'` T(s,a,s') [R(s,a,s') + `gamma` V(s')]. Use el modelo suministrado, incluyendo los rebotes y la recompensa terminal al entrar.

c) Complete el bloque de `value_iteration` desde V0(s)=0. Use barridos sincrónicos: todos los valores nuevos se calculan con el diccionario del barrido anterior. Mantenga V(terminal)=0. Calcule `delta` como el mayor cambio absoluto entre barridos y `converged` como `delta` < `tolerance`. El ciclo suministrado termina al converger o alcanzar `max_iterations` y guarda los resultados para la interfaz.

d) Use `control` como caso básico para comprobar la implementación. En `cuadricula` observe cómo se propagan los valores durante los primeros barridos hasta converger. Describa lo observado y el criterio de parada en el literal d del punto 6a.

Punto 5: Extracción y análisis de políticas (15 %)

Una vez calculados los valores de los estados, el dron necesita seleccionar una acción en cada posición. La política debe considerar los posibles resultados de cada movimiento, no solo el valor de la celda vecina.

a) Implemente `extract_policy` en `algorithms/planning.py`. Para cada estado no terminal escoja la acción de mayor `action_value` y conserve la primera en el orden N, E, S, W en caso de empate. Retorne `None` para los terminales.

b) En `atajo` compare la política con ruido 0 y 0.3, manteniendo los demás parámetros. Identifique un cambio de decisión para explicarlo en el literal e del punto 6a.

c) En `zona_riesgo` compare `living_reward`=-0.04 y -0.30. Por separado, compare `discount`=0.50 y 0.95 con los demás parámetros originales. Observe el efecto sobre los valores y la política y conserve un ejemplo representativo para el literal e del punto 6a.

Punto 6: Reflexión sobre el trabajo realizado (10 %)

Organice el PDF por puntos e incluya los cálculos, resultados y explicaciones solicitados. Responda las siguientes preguntas de forma concisa, apoyándose en ejemplos, tablas pequeñas o capturas seleccionadas. Cuando corresponda, haga referencia a las respuestas anteriores en lugar de repetirlas.

6a. Análisis de los algoritmos y resultados (6 %)

a) ¿Cómo cambian la probabilidad de incendio y la decisión al recibir evidencia? Distinga probabilidad y utilidad esperada.

b) ¿Por qué enumeración y eliminación deben coincidir? ¿Qué efecto observó al cambiar el orden de eliminación?

c) ¿Cómo influyen los intentos y la probabilidad de la evidencia en el error y la cantidad de muestras aceptadas?

d) ¿Qué muestran los barridos de valores y qué criterio se usó para detenerlos?

e) ¿Qué efecto observó al variar el ruido, la recompensa por movimiento y el descuento? Explique un cambio representativo en los valores o en la política y distinga una política de una ruta fija.

6b. Reflexión sobre el uso de inteligencia artificial (4 %)

Escriba una reflexión concisa sobre el uso de IA durante el desarrollo. Explique qué aprendió al revisar sus sugerencias, qué cambios aceptó o rechazó, cómo verificó la solución y si podría explicarla en una sustentación. Si no utilizó IA, explique por qué.

Política de uso de inteligencia artificial

Primero desarrolle una versión autónoma de la solución. Después podrá usar IA para apoyar su revisión, corrección, depuración, explicación o mejora. Para cada función trabajada con IA, incluya en los anexos del PDF la versión inicial, los prompts, las sugerencias relevantes, los cambios aceptados, rechazados o adaptados y la forma de verificación. Mantenga únicamente la versión final activa en el código.

Entrega del trabajo

Entregue un archivo .zip con el proyecto y el PDF de análisis, incluidos sus anexos. Incluya nombres y códigos de los integrantes. Un integrante del grupo debe subirlo al espacio correspondiente en Bloque Neón dentro de la fecha y hora indicadas. No modifique las firmas ni los formatos de retorno definidos por el proyecto.

Al entregar, declara que las versiones iniciales son de su autoría, que comprende el código final y que el equipo docente puede revisarlo, ejecutarlo y solicitar su explicación.